深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 10 — Text Data and NLP 第十讲·文本数据与自然语言处理
从词袋到词向量，从分词到大模型

任课教师：戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University

上课时间：第 3–14 周 · 星期四 11-12 节

上课地点：四方楼南 209

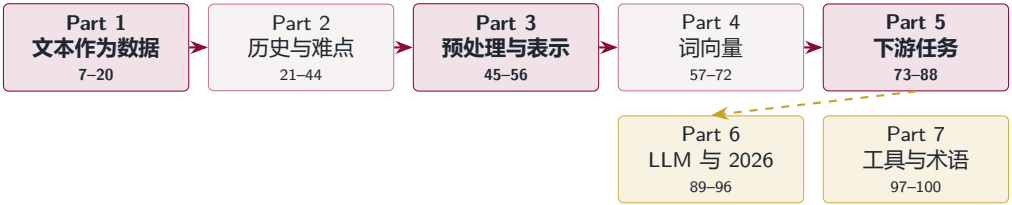
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 把非结构化文本变成可分析的矩阵（分词、TF-IDF、n-gram）；
2. 解释词向量 word vector 为什么能表示语义，并训练一个；
3. 用word2vec / GloVe / 共现矩阵三种视角理解嵌入；
4. 做情感分类、主题模型、命名实体识别并选对评价方式；
5. 用 Cohen's kappa 核验 LLM 标注的一致性；
6. 用 RAG 与引用溯源把大模型接到自己的语料上。

来源：对应课程大纲第 10 讲目标：把非结构化文本转化为可分析数据；掌握情感、主题、实体抽取与 LLM 时代的文本方法。

本讲路线图 Roadmap（100 帧）



主线：文本作为数据 → 历史与难点 → 预处理与表示 → 词向量 → 下游任务 → LLM 实践

代码页 10 帧•样本页 14 帧；中英双语句料，术语全程中英对照

与 L8/L9 的衔接 From Embeddings to Text

L8 给了什么

嵌入与向量检索（BERTopic、FAISS），但没有回答" 这些向量是从哪来的"。

L9 给了什么

Transformer 与预训练-微调。本讲把它接到文本上：怎么把句子变成 token、再变成向量。

本讲要补的是文本特有的东西：分词、词形、n-gram、TF-IDF、词向量的训练与评估、序列标注与主题模型。这些在表格与图像里都不存在。

来源：衔接说明为本讲组织。

本讲输出物 Deliverables

1) 预处理管线	2) 11 个程序	3) 术语表
中文分词 + 领域词典 + 停用词 + n-gram，可直接复用到项目语料。	分词、TF-IDF、词向量、共现/SVD、情感、NER、主题、标注一致性、RAG、词典。	81 条中英对照（CSV + Word/Excel + L6–L10 合并版）。

语料全部合成（评论、合同、主题文本，中英各半），种子固定 20261111。

来源：程序与语料随课程仓库发布。

全讲导览 Map of the Lecture

部分	帧	关键问题
文本作为数据	7–20	文本能测什么？有什么坑？
历史与难点	21–44	NLP 怎么走到今天？为什么难？
预处理与表示	45–56	中文怎么切词？怎么变成矩阵？
词向量	57–72	语义怎么变成几何？怎么训练与评估？
下游任务	73–88	情感、主题、实体、摘要、问答各怎么做？
LLM 与 2026	89–96	大模型怎么标注？怎么接自己的语料？
工具与术语	97–100	用什么库？术语怎么背？

读法：每个主题都走一遍直觉 → 形式化 → 样本图 → 代码 → 评价。

来源：导览为本讲安排。

为什么文本是”数据” Why Text Is Data

文本的三个特点

- 非结构化：没有行列；
- 高维稀疏：词表几万，每句只用到几十个；
- 语义依赖上下文：同一个词在不同语境意思不同。

所以要做两件事

1. 表示：把文本变成向量（本讲的核心）；
2. 评价：文本任务的正确性往往需要人工判断。

一句话：文本分析 = 表示 + 模型 + 评价。三者缺一，结论都站不住。

来源：三个特点据课程大纲与 Jurafsky & Martin；三件事的提法为本讲组织。

四类研究问题 Four Uses of Text as Data

用途	测什么	典型数据
宏观不确定性	经济政策不确定性 economic policy uncertainty、地缘风险	新闻、政策文件
情绪与语气	正负面、情绪强度	评论、社交媒体
财报文本	风险提示、管理层语气	年报、MD&A
研报与分析师	观点、目标价、分歧	券商报告

课程讲义的第一部分就是这四类（源稿第 3–8 页）。共同点：先把文本变成指标，再做计量或预测。

来源：四类用途据课程讲义；术语为本讲补充。

宏观不确定性与风险 Measuring Uncertainty

做法

用词表 + 频率构造指数：统计新闻中" 不确定""风险""政策" 等词的占比，按月份聚合。

要点

- 词表要领域定制（中文尤其如此，帧 95）；
- 要做归一化（除以总词数）；
- 与宏观变量做相关或预测（衔接 L12 的因果）。

经典例子是 Baker-Bloom-Davis 的经济政策不确定性指数。它的价值不在" 多准"，而在提供了一个可复现、可更新的指标。

来源：EPU 指数据 Baker, Bloom & Davis (2016)；说明为本讲补充。

情绪与语气：从词典到监督学习

词典法

给词表打正负分，句子里求和。便宜、可解释，但否定与反讽处理不好。

监督学习

用标注数据训练分类器（帧 75）。更好，但要标注；且换了领域就要重标。

本讲的做法：两种都跑，用同一套指标比（帧 76）。这也是项目报告里应该给的"基线对照"。

来源：两种方法据课程讲义与实践；对照要求与 L5 的评估协议一致。

财报与研报：结构化与非结构化互补

财报里能挖什么

风险提示的措辞变化、管理层讨论的语气、" 不确定性" 词汇的密度——这些表格里没有。

与表格数据的关系

表格给" 结果"，文本给" 原因与预期"。两者结合往往比单独用任何一个更准（衔接 L7 的集成）。

实务提醒：财报文本有固定格式（模板化），所以纯文本特征容易学成" 模板识别"，要小心构造对照组。

来源：实务提醒为本讲补充；与 L7 的集成思路一致。

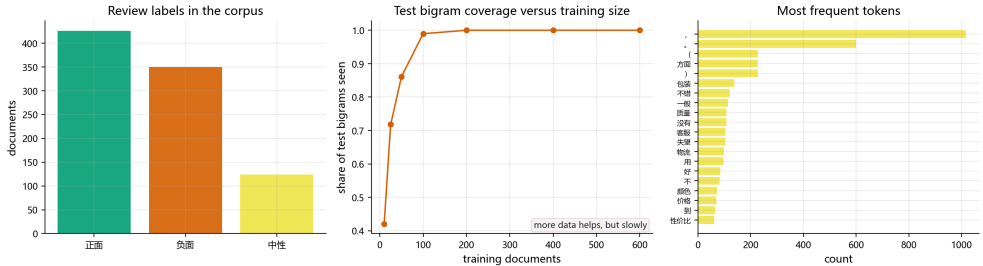
文本数据的四个特点 Four Properties

稀疏	高维	噪声	隐私
词表大、单词出现少；矩阵 99% 是零。	几万维很常见，降维与正则必不可少。	错别字、表情、广告、复制粘贴。	文本里最容易混入个人信息，需脱敏。

四点里最容易被忽略的是隐私：一条评论里可能包含姓名、电话、住址。项目里如果抓了平台评论，交付前必须做脱敏与合规检查。

来源：四特点据实践；隐私提醒为本讲补充（衔接帧 16）。

【样本 + 可视化】语料长什么样



左：本讲合成语料的标签分布（正面 / 中性 / 负面，含 12% 的标注噪声）；中：测试集 bigram 覆盖率随训练语料增长——从 42% 升到 100%（合成语料词表小，真实语料会更平缓）；右：最高频的 20 个 token。

程序 `110_01_preprocess.py` ·语料 `110_reviews.csv`（900 条，中英各半）

从文本到特征：一条流水线 The Text Pipeline



每一步都可能出错

分句错 → 语义断；分词错 → 特征错；表示错 → 后面全错。

所以

文本项目的时间，大半花在这条管线上，而不是模型上。

来源：流水线为本讲组织；时间分配的观察据实践。

评价文本任务：为什么准确率不够

三个原因

1. 标注本身有主观性（" 这条评论算负面吗"）；
2. 类别不平衡（负面通常少数）；
3. 同一个意思有多种写法，指标对措辞敏感。

所以要看

每类 F1、混淆矩阵、错误样例，以及标注一致性（帧 86、90）。

报告写法：给出指标 + 给出错误样例 + 给出标注协议。只报一个 0.93 的准确率，评审无法判断可信度。

来源：评价困境据实践与课程讲义；报告要求为本讲补充。

隐私与合规 Privacy and Compliance

三类风险

- 个人信息（姓名、电话、住址）；
- 商业秘密（未公开的合同条款）；
- 版权（转载的正文）。

三条做法

1. 脱敏：规则 + 模型识别后替换；
2. 最小化：只保留分析必需的字段；
3. 本地化：敏感语料用本地模型，不传第三方 API。

这一节直接对应评分表的" 数据获取与合规"。在项目四（合规平台）里，文本脱敏本身就是交付物的一部分。

来源：三条做法据实践；与项目四的关系为本讲补充。

中文与英文：处理上的四个差别

维度	英文	中文
切词	空格分隔	必须分词（帧 46）
词形	需词干/词形还原	基本不需要（无屈折）
字符	ASCII	全半角、繁简、生僻字
未登录词	拼写变体	专有名词与新品名（帧 95）

所以"中文 NLP 更难"的说法不准确——难的地方不同：英文难在形态，中文难在切词与未登录词。

来源：四个差别据 Jurafsky & Martin 与中文处理实践；说法为本讲补充。

语料从哪来 Where Corpora Come From

来源	优点	风险
公开语料	有许可、可复现	领域不一定匹配
平台 API	规模大、更新快	条款限制、随时变更
自建标注	完全贴合任务	贵、慢，需一致性检验
合成数据	便宜、可控	与真实分布有差距

本讲用合成语料是为了可复现与公开；你在项目里用真实语料时，必须写清来源、许可、时间范围与脱敏方式。

来源：来源与风险据实践；合成语料的用途说明为本讲补充。

工程现实：文本管道比模型更费时

四个耗时点

1. 编码与乱码（GBK / UTF-8 / BOM）；
2. 分句与清洗（HTML、表情、全角）；
3. 领域词典维护；
4. 标注与一致性检查。

建议

把管线写成可重跑的脚本（本讲的 11 个程序就是），并把中间结果存成 CSV——便于复查与复现。

这也是" 可复现性" 在文本项目里的具体含义：别人拿到你的脚本与语料，能跑出同样的数字。

来源：四个耗时点据实践；建议与 L5 的可复现要求一致。

小结：文本分析的三件事

表示

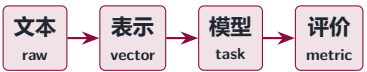
分词、归一化、TF-IDF、词向量——本讲的主线。

评价

指标 + 错误样例 + 标注一致性，三者一起报。

模型

分类、聚类、序列标注、生成——按任务选。



下一部分：NLP 是怎么走到今天的，以及它为什么难。

来源：小结为本讲整理。

历史（一）：符号时代 Symbolic NLP

做法

手写语法规则、词典、模板；代表任务是规则翻译与专家系统。

优点与代价

可解释、可控；但脆弱——换一个领域就要重写规则，覆盖率上不去。

符号时代的遗产：语言学知识（词性、句法）至今仍用于特征工程、规则抽取与标注规范。

来源：历史分期据课程讲义第 9–14 页与 Jurafsky & Martin。

历史（二）：统计时代 Statistical NLP

转折

有了语料库，就可以估计概率：n-gram 语言模型、HMM 词性标注、统计机器翻译。

关键变化

从"写规则"变成"估参数"；代价是需要标注语料，且稀疏问题突出（帧 33）。

这一时代的核心工具是最大似然 + 平滑——本讲帧 50 的 n-gram 就是它的最小版本。

来源：统计时代据课程讲义与 Jurafsky & Martin；平滑的作用为本讲补充。

历史（三）：神经时代 Neural NLP

三步

词向量（2013）→ 预训练语言模型（2018）→ 大模型（2020 后）。每一步都在减少人工特征。

为什么能成

数据、算力、可迁移的表示（衔接 L9 帧 38）。

三次范式的共同规律：把" 人写的规则" 换成" 从数据里学的表示"。本讲后半部分讲的词向量，就是这条路的起点。

来源：分期据课程讲义第 9–14 页；规律归纳为本讲补充。

四个经典任务 Four Classic Tasks

会话代理	问答	机器翻译	信息抽取
意图识别 + 槽位填充；规则 → 模型 → LLM。	检索 + 阅读;今天的主流是 RAG (帧 92)。	从规则到统计到神经；BLEU 是常用指标。	实体、关系、事件；NER 是基础 (帧 77)。

四个任务的共同骨架：把语言映射成结构。结构可以是意图、答案、译文或实体表——本讲后面的任务都是这个骨架的变体。

来源：四个任务据课程讲义第 15–20 页；骨架归纳为本讲补充。

语言学层级 Six Levels of Linguistic Knowledge

层级	研究对象	对应任务
音系学 phonetics and phonology	声音	语音识别
形态学 morphology	词的内部结构	分词、词形
词性标注 part-of-speech tagging	词类	POS、句法前置
句法 syntax	词如何组句	依存分析、NER 的上下文
语义学 semantics	意义	情感、词义消歧
语篇 discourse	跨句结构	指代消解、摘要

不必成为语言学家，但要知道每一层会引入什么麻烦——下一帧给每层配一个商务例子。

来源：六层据课程讲义第 21–28 页与 Jurafsky & Martin。

音系与形态：从声音到词素

音系学

语音的区分与变体；对文本分析影响小，对语音输入影响大。

形态学

词由词素组成：work / worked / working。中文的" 字-词" 关系是分词的核心（帧 46）。

商务例子：" 华为 Mate60Pro" 是一个词还是四个词？分词器的默认答案往往错——所以要领域词典（帧 95）。

来源：例子为本讲补充；义项划分据 Jurafsky & Martin。

词性与句法：为什么 POS 有用

词性标注

给每个词标" 名词/动词/形容词"。用处：否定词（不、没）与修饰关系的判断依赖词性。

句法与依存

" 降价" 是主语还是宾语？依存分析给出谁对谁做了什么——抽取合同条款的关键。

一个经典坑：" 不满意" = 负面，但" 满意" 是正面。不去看否定词，情感分类会把整句判反。

来源：否定词问题据实践；依存的作用为本讲补充。

语义与语篇：词义与指代

语义

词义（" 便宜" 是褒是贬?）、句义、一词多义（帧 71）。

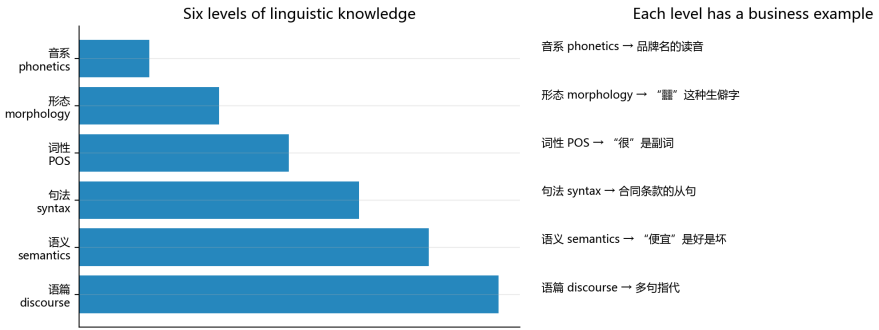
语篇

跨句指代（" 它" 指什么）、篇章结构（论点-论据）。摘要与问答都依赖这一层。

商务例子：合同里" 乙方应在 上述 期限内……"——" 上述" 指的是哪一段，只有语篇层能回答。

来源：例子为本讲补充；层级划分据课程讲义。

【样本 + 可视化】六个层级，各配一个商务例子



左：六个层级的阶梯（从声音到语篇）；右：每个层级对应的商务问题——品牌名怎么读、生僻字怎么切、"很"是什么词性、合同从句怎么解析、"便宜"是好是坏、多句指代指的是谁。

程序 `110_01_preprocess.py` ·图为本讲自制

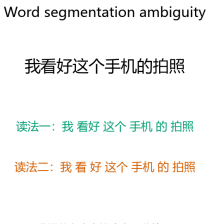
歧义：为什么一句话有多种读法 Ambiguity

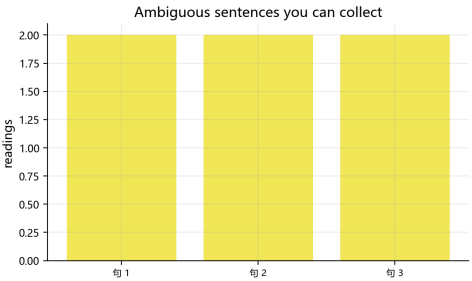
词义歧义	句法歧义	指代歧义
" 苹果" 是水果还是公司?	" 我看见拿着望远镜的人" ——谁拿望远镜?	" 小明和小刚吵架，他先动手"——他是谁?

歧义不是" 难一点"，而是本质上多解。所以 NLP 系统的输出往往需要 ** 上下文或外部知识 ** 来消歧——这就是为什么大模型带来了质变（帧 89）。

来源：三类歧义据课程讲义第 31–34 页；推论为本讲补充。

【样本 + 可视化】分词歧义会直接改变特征





"我看好这个手机的拍照"有两种切法："我看好 / 这个 / 手机"（我看好它）与"我 / 看 / 好 / 这个 / 手机"（我看这个好手机）。切法不同，情感特征完全不同——分词错误会一路传到模型。

程序 `l10_01_preprocess.py` ·分词示例见 `l10_segmentation_demo.csv`

规模与稀疏：语言是离散无限的

规模

语言组合几乎无限：20 个词的句子有无数种可能。所以任何语料都只是样本。

稀疏

语料越大，见过的组合越多，但长尾永远存在——这就是 n-gram 需要平滑的原因（帧 50）。

本讲实测：合成语料上，测试集 bigram 覆盖率随训练语料从 42% 升到 100%（词表小）；真实语料会停在 60–80% 附近，永远到不了 100%。

来源：稀疏问题据课程讲义第 49–52 页；实测数字来自程序 l10_01。

变体：模型永远落后于语言 Variation

四种变体

同义表达、方言、错别字、网络新词。每一种都让词表失配。

三种应对

1. 归一化（繁简、全半角、英文大小写）；
2. 子词（subword）切分，让未见过的词也能表示；
3. 领域词典与规则兜底（帧 95）。

子词切分（BPE/WordPiece）是 2016 年后的标准做法：把词切成更小的单位，从而缓解未登录词问题。

来源：子词方法据 Sennrich et al. (2016)；三种应对为本讲整理。

表达力与”没进模型的东西”

表达力

语言能表达否定、假设、反讽——这些在词袋里完全丢失（” 不差” 和” 差” 词袋几乎一样）。

未建模变量

说话人、时间、场合、语气……模型没显式放进来的东西，它会悄悄忽略，但你可能会误读结果。

务实做法：在报告里写清” 这个模型忽略了什么”——比声称” 我们的 NLP 模型理解语义” 要诚实得多。

来源：表达力问题据课程讲义第 55–58 页；务实建议为本讲补充。

两次范式转换 Two Paradigm Shifts



转换的驱动力

数据规模、算力、以及表示方法的突破（词向量）。

没有消失的东西

语言学知识仍在：标注规范、评测集设计、错误分析。

来源：时间线据课程讲义；归纳为本讲补充。

什么改变了 NLP 的版图 What Changed the Landscape

数据	算力	表示
万亿 token 的语料，让预训练成为可能。	GPU/TPU 让大模型训练可行（衔接 L9 帧 11）。	从词向量到上下文嵌入，一个模型适配多任务。

课程讲义把这三点列在一起。它们的关系是：表示方法让数据与算力"变现"——没有好的表示，再多数据也学不动。

来源：三点据课程讲义第 66 页；关系说明为本讲补充。

NLP 与机器学习、语言学的关系

领域	提供什么	本讲用在哪
机器学习	优化、泛化、评估	分类器与指标（帧 75）
语言学	结构、层级、标注规范	词性/句法/语篇（帧 25–28）
信息检索	TF-IDF、向量空间	表示与检索（帧 52、92）
统计学	概率模型、平滑	n-gram 与 LDA（帧 50、81）

NLP 是交叉学科：一个文本项目往往要同时用到这四样。本讲的编排也按这个顺序：先表示（IR/统计），再模型（ML），后结构（语言学）。

来源：交叉关系据课程讲义第 62–64 页；编排说明为本讲补充。

符号、概率与连接主义 Symbolic, Probabilistic, Connectionist

符号	概率	连接主义
规则清晰、可审计；不鲁棒。	能处理不确定性；需要语料。	表示学习、端到端；黑箱、吃数据。

今天的主流是混合：用规则做兜底与合规（项目四），用模型做泛化（分类、抽取），用 LLM 做少样本（帧 89）。选哪种取决于数据量与可解释要求。

来源：三派划分据课程讲义第 60–62 页；混合实践为本讲补充。

方法地图：四种做法各自适合什么

做法	需要	适合	代价
规则/词典	领域知识	抽取、合规	维护成本
TF-IDF + 线性	少量标注	分类基线	低
词向量 + 浅层	语料	中小规模分类	中
预训练 + 微调	GPU、标注	复杂任务	高

顺序建议：规则 → TF-IDF → 词向量 → 预训练，每一步都先看基线够不够。本讲帧 43 给出成本-效果的示意。

来源：方法地图为本讲整理；顺序建议与 L7/L9 一致。

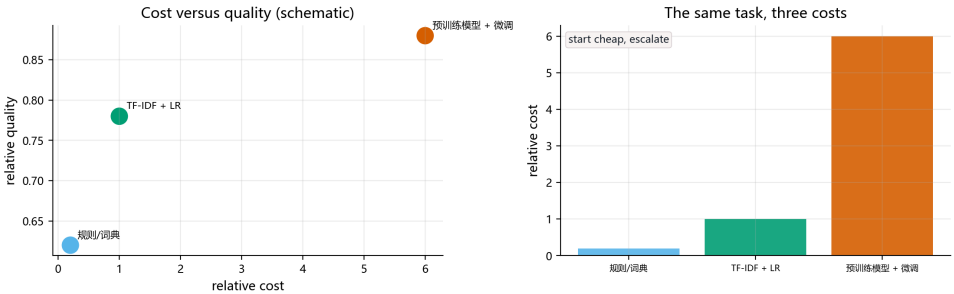
对文本模型的四个期望 Desiderata

可解释	可迁移	鲁棒	可评估
为什么判成负面？要能说出依据。	换领域还能用（词向量、预训练的价值）。	错别字、表情、对抗输入不应崩。	有明确的指标与标注协议。

四个期望互相拉扯：可解释往往要牺牲一点精度，鲁棒往往要更多数据。报告里说清你优先保了哪个。

来源：四个期望据课程讲义第 59 页；权衡说明为本讲补充。

【样本 + 可视化】同一个任务，三种做法的成本与效果



左：规则/词典、TF-IDF + 逻辑回归、预训练 + 微调的相对成本与相对效果（示意图）；右：成本对比。
从便宜的方案开始，不够再升级——这是本讲与 L7、L9 一致的工程纪律。

程序 `l10_02_tfidf_baseline.py` ·数字为教学示意，非真实基准

语料库：统计 NLP 的燃料 Corpora

类型

通用语料（新闻、百科）、领域语料（财报、合同）、标注语料（带标签）。

质量检查

去重、去模板、检查标注一致性；语料质量比模型选择更影响结果。

一个具体的坑：同一文档的不同片段被分到训练与测试——这是文本特有的泄漏（帧 87）。

来源：语料类型据课程讲义第 46 页；质量检查为本讲补充。

小结：NLP 难在歧义、稀疏、变体

歧义	稀疏	变体
一句话多种读法，需要上下文。	组合无限，语料永远不够。	语言在变，模型追不上。

这三难解释了 NLP 的全部方法演进：词向量用稠密空间缓解稀疏，上下文嵌入缓解歧义，预训练缓解变体。下一部分讲第一步：怎么把文本变成矩阵。

来源：小结为本讲整理。

过渡：从”读懂”到”变成矩阵”

我们已经知道

文本难在歧义、稀疏、变体；评价要看一致性；方法有四种档位。

接下来

预处理（帧 45–56）与表示（帧 57–72）——这是文本分析的" 第一公里"。



来源：过渡为本讲组织。

预处理管线 The Preprocessing Pipeline



每一步都要留证据

保留原始文本与处理后的文本两列；出错时才能回溯是哪一步坏的。

别过度清洗

表情符号对情感分析很有用；去掉否定词会毁掉情感任务（帧 49）。

来源：管线与注意事项据实践；本讲程序 l10_01 实现了这条线。

中文分词 Word Segmentation

三种算法

1. 最大匹配：按词典贪心地切最长词；
2. 统计模型：HMM / CRF，把分词当序列标注；
3. 混合：词典 + 模型（jieba 的做法）。

两个难点

- 歧义切分：" 研究生命起源"；
- 未登录词：新品名、人名、缩写。

本讲用 jieba：它用前缀词典 + 动态规划找最大概率路径，并支持 `add_word` 注入领域词（帧 95 实测 2 → 5 个术语保住）。

来源：算法据 jieba 文档与 Jurafsky & Martin；实测来自 l10_10。

【代码】中文分词、停用词与 n-gram

```python
import jieba                                    # Chinese word segmentation
stopwords = set("的 了 也 很 都 就 和 与 在 是 我 有 这 那".split()) # a small stop list
def tokenize(text, use_lexicon=False):          # tokenise one Chinese sentence
    if use_lexicon:                             # add the domain terms first
        for term in ["华为Mate60Pro", "比亚迪海獭", "vivoX100", "李宁䨻科技"]: # product names
            jieba.add_word(term)                # teach the segmenter the term
    return [t for t in jieba.lcut(text) if t.strip() and t not in stopwords] # cut and drop stopwords
samples = make_lexicon_sentences()              # sentences with domain terms
for s in samples:                               # loop over the sentences
    plain = " / ".join(jieba.lcut(s))           # default segmentation
    print(s, "->", plain)                       # report the cut
def cut(text):                                  # tokenise for the vectoriser
    if any("\u4e00" <= ch <= "\u9fff" for ch in str(text)): # contains Chinese characters
        return " ".join(t for t in jieba.lcut(str(text)) if t.strip()) # segment with jieba
    return " ".join(str(text).lower().split())  # otherwise split on whitespace
```

实测：默认词典只保住 2 个领域术语，加入词典后保住 5 个。逐行注释的完整程序 `l10_01_preprocess.py` 与 `l10_10_domain_lexicon.py`。

归一化：词干、词形与统一字形

操作	例子	适用
词干提取 stemming	studies → studi	英文检索
词形还原 lemmatisation	studies → study	英文句法
大小写	Apple → apple	英文通用
全半角	Ａ Ｂ Ｃ → ABC	中文预处理
繁简	臺灣 → 台湾	中文通用

中文不需要词干与词形（没有屈折变化），但需要全半角、繁简、异体字统一。这是中英管线的最大差异之一。

来源：操作与适用据 Jurafsky & Martin 与中文处理实践；对比为本讲补充。

停用词：什么时候该去，什么时候不能去

该去

主题建模、词袋分类：去掉" 的、了、是" 能降维、减少噪声。

不能去

否定词（不、没、无）、程度词（很、太）、转折词（但是、不过）——它们是情感与逻辑的载体。

本讲的停用词表只有十几个词（" 的了也很……"），故意保留" 不" 和" 没"——因为情感任务里它们决定正负。

来源：停用词的取舍据实践；本讲词表见 l10_01。

n-gram 与平滑 n-grams and Smoothing

定义

把连续的 n 个词当作一个单位：

$$P(w_t \mid w_{t-n+1:t-1}) \approx \frac{\text{count}(w_{t-n+1:t})}{\text{count}(w_{t-n+1:t-1})}.$$

为什么需要平滑

没出现过的组合概率为 0，一句里有一个就整句为 0。加一平滑（Laplace）或插值解决。

n 越大越准但越稀疏；通常 $n = 2$ 或 3。本讲实测：测试集 bigram 覆盖率 42% → 100%（合成语料），真实语料停在 60–80%，剩下的就是需要平滑的部分。

来源：公式据 Jurafsky & Martin；实测来自 l10_01。

词袋模型 Bag of Words

定义

每篇文档变成一个向量，第 i 维是第 i 个词的出现次数。

$$X \in \mathbb{R}^{N \times |V|}.$$

丢掉了什么

词序、语法结构、否定范围。"狗咬人"与"人咬狗"向量一样。

但别小看它：在短文本分类上，词袋 + 线性模型常常是极强的基线（帧 55）。它会输在长文本、需要词序的任务上。

来源：词袋定义据 Manning et al.；评价为本讲补充。

TF-IDF：压掉高频虚词

公式

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \log \frac{N}{\mathrm{df}(t)},$$

tf 是词在文档中的频次，df 是包含该词的文档数。

直觉

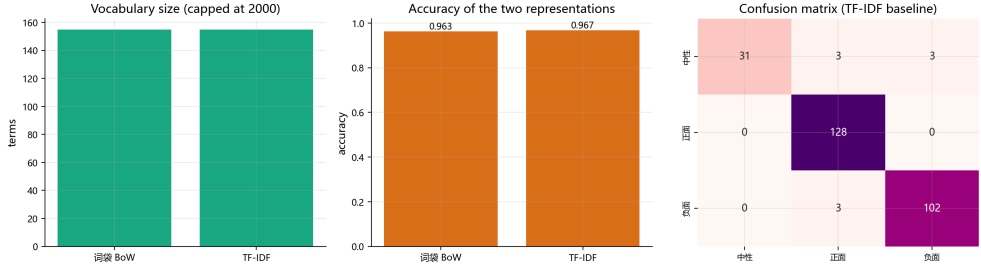
一个词在很多文档里出现（如" 的"），说明区分度低，权重被压低；只在少数文档里出现（如" 退款"），权重被抬高。

两个细节

1. 用 log 压频次，避免长文档主导；
2. 归一化（L2）后余弦相似度才可比。

来源：公式与直觉据 Manning et al., *Introduction to Information Retrieval*。

【样本 + 可视化】词袋与 TF-IDF 几乎打平



左：词表大小（都截到 2000 维）；中：词袋准确率 0.963、TF-IDF 0.967；右：TF-IDF 模型的混淆矩阵。

差别很小——说明在这份语料上，表示不是瓶颈，标注噪声才是。

程序 `l10_02_tfidf_baseline.py` ·对照表 `l10_baseline_scores.csv`

稀疏表示与哈希技巧 Sparsity and Hashing

稀疏

词表几万维，单文档只用到几十维——矩阵 99% 是零。所以用稀疏矩阵存（scipy.sparse）。

哈希技巧

不用词表，用哈希函数把词映射到固定维度。省内存，但不可解释、会碰撞。

本讲实测：共现矩阵是 60×60，非零占比 9.6%；真实语料的文档-词矩阵通常 $<1\%$。稀疏是文本数据的常态。

来源：稀疏与哈希据 Manning et al. 与 Weinberger et al. (2009)；实测来自 l10_04。

【代码】TF-IDF + 逻辑回归基线

```python
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer # the two representations
from sklearn.linear_model import LogisticRegression   # the baseline classifier
for name, vec in [("词袋 BoW", CountVectorizer(max_features=2000)), # raw counts
                  ("TF-IDF", TfidfVectorizer(max_features=2000))]: # tf-idf weights
    vec.fit(X_tr)                                     # fit on the training split only
    Atr = vec.transform(X_tr)                         # training matrix
    Ate = vec.transform(X_te)                         # test matrix
    model = LogisticRegression(max_iter=2000).fit(Atr, y_tr) # the baseline classifier
    pred = model.predict(Ate)                         # test predictions
    acc = float(accuracy_score(y_te, pred))           # accuracy
    f1 = float(f1_score(y_te, pred, average="macro"))  # macro F1
    rows.append({"representation": name, "vocab": len(vec.vocabulary_), # the row
                 "nnz_per_doc": round(float(Atr.nnz / Atr.shape[0]), 1), # sparsity
                 "accuracy": round(acc, 4), "macro_f1": round(f1, 4)}) # the scores
vec, model, Ate, pred = matrices["TF-IDF"]            # use the TF-IDF model
names = np.array(vec.get_feature_names_out())          # the vocabulary
```

实测：词袋 0.963 / TF-IDF 0.967（宏 F1 0.947 / 0.953）。永远先跑这个基线——它比大多数复杂模型便宜得多。完整程序 `l10_02_tfidf_baseline.py`。

小结：表示决定上限

三句话

分词是中文的第一道坎；TF-IDF 是便宜而强的基线；稀疏是常态，别当异常。

局限

词袋丢词序、丢否定、维度高、无相似度概念——所以需要稠密向量。



下一部分：让" 相似" 变得可计算——词向量。

来源：小结为本讲整理。

离散符号的三个问题 One-Hot and Its Problems

one-hot

词表 $|V| = 10^4$，每个词是一个 10^4 维向量，只有一位是 1。

三个问题

1. 维度爆炸：$10^4 \times 10^4$ 的参数；
2. 正交：任意两个词的余弦相似度都是 0；
3. 无语义：猫和狗的距离与猫和飞机一样。

WordNet 一类的专家词典试过解决第 3 点，但：词义更新慢、覆盖不全、无法表示" 像的程度"。这就是词向量的动机。

来源：三个问题据课程讲义第 78–79 页；WordNet 批评据第 75–77 页。

分布假设 The Distributional Hypothesis

一句话

一个词的意思，由它常出现的上下文决定。

例子

”退款”“退货”“赔付”常出现在相似的句子里（”我要申请 ××”“客服不给 ××”）——所以它们应该相似。

怎么变成算法

统计共现（帧 63），或用上下文预测（帧 59）。两条路殊途同归，GloVe 把它们结合（帧 65）。

这是本讲最重要的一句话：语义 = 分布 = 几何。后面所有词向量方法都是它的实现。

来源：分布假设据 Harris (1954) 与 Firth (1957)；表述为本讲组织。

word2vec：用上下文预测 Word2vec Objective

skip-gram 的目标

对每个位置 t 与窗口内的 j：

$$\max \sum_{t} \sum_{-m \le j \le m,\ j \neq 0} \log P(w_{t+j} \mid w_t),$$

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_w \exp(u_w^\top v_c)}.$$

两个矩阵

v（中心词）与 u（上下文）各一套参数；训练完通常把 $u + v$ 或只用 v 当词向量。

注意：这个 softmax 的分母要遍历整个词表，所以大词表下算不动——这就是负采样要解决的问题（帧 61）。

来源：目标函数据 Mikolov et al. (2013a)；复杂度说明为本讲补充。

训练：随机梯度下降 Train with SGD

每次更新

对一个 (中心词, 上下文) 对，把 $\log P$ 对参数求导，走一小步。批次通常就是一个窗口。

参数量

$2|V|d$。本讲 $|V| = 185$、$d = 60 \to$ 22,200 个参数，在小语料上几秒训完。

工程上的三个旋钮：窗口（语义范围）、维度（表达能力）、负样本数（训练效率）。帧 68–69 会看到它们如何影响评估结果。

来源：训练细节据 Mikolov et al. (2013a)；参数量为本讲实测。

负采样 Negative Sampling

想法

把" 从 $|V|$ 类里挑一个" 变成" 这是真邻居吗" 的二分类：真邻居 (正例) + 几个随机词 (负例)。

噪声分布

负例按词频的 $3/4$ 次方 采样：

$$P(w) \propto \text{freq}(w)^{3/4},$$

让高频词更容易被抽到，但不像纯频率那样极端。

好处：每次更新只算 $k+1$ 个词（k 通常 5–15），而不是整个词表。这是 word2vec 能在大语料上跑的关键。

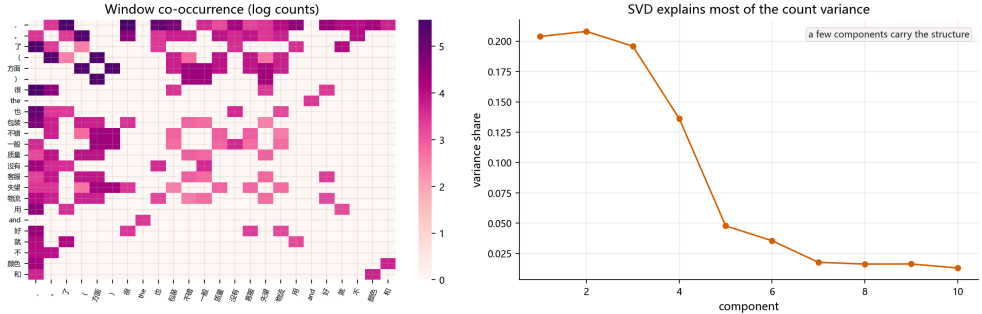
来源：负采样据 Mikolov et al. (2013b)；$3/4$ 的由来与效果见原文。

【代码】用 gensim 训练词向量

```python
from gensim.models import Word2Vec                    # the word-vector model
sentences = [cut(t) for t in reviews["text"]]         # one token list per review
model = Word2Vec(sentences=sentences, vector_size=60, window=4, min_count=1, # small vectors for speed
                 sg=1, negative=5, epochs=80, seed=20261111, workers=1) # skip-gram with negative sampling
print("vocabulary:", len(model.wv), "| vector size:", model.wv.vector_size) # report
for word in probes:                                   # loop over the probes
    if word not in model.wv:                          # skip words outside the vocabulary
        continue                                      # next probe
    neighbours = model.wv.most_similar(word, topn=5)  # the five nearest words
    rows.append({"word": word, "neighbours": " / ".join("%s(%.2f)" % (w, s) for w, s in neighbours)}) # record
for a, b, c, d in make_analogy_pairs():               # loop over the analogy triples
    guess, score = model.wv.most_similar(positive=[b, c], negative=[a], topn=1)[0] # a - b + c
    analogy_rows.append({"a": a, "b": b, "c": c, "expected": d, "got": guess, # the record
                         "score": round(float(score), 3), "hit": guess == d}) # whether it matched
```

实测（词表 185、维度 60）：发货 → 速度 (0.92) / 第二天 (0.90) / 快 (0.86)；退货 → 态度 (0.90) / 方便 (0.90)。完整程序 `110_03_word2vec.py`。

【样本 + 可视化】共现矩阵：计数也能得到向量



左：窗口共现矩阵的对数计数（60×60，非零占比 9.6%）；右：截断 SVD 的方差解释——前几个成分就承载了主要结构。这就是 LSA/COALS 的思路：计数 + 降维 = 词向量。

程序 `110_04_cooccurrence.py` ·矩阵 `110_cooccurrence.csv`

【代码】共现矩阵与截断 SVD

```
window = 2                                          # context window on each side
X = np.zeros((len(vocab), len(vocab)), dtype=float)   # the co-occurrence matrix
for doc in docs:                                    # loop over the documents
    for i, word in enumerate(doc):                  # loop over the positions
        if word not in index:                       # skip out-of-vocabulary tokens
            continue                                # next token
        for j in range(max(0, i - window), min(len(doc), i + window + 1)): # the context window
            if j == i:                              # skip the centre token
                continue                            # next position
            if doc[j] in index:                     # keep only vocabulary tokens
                X[index[word], index[doc[j]]] += 1.0     # increment the count
svd = TruncatedSVD(n_components=10, random_state=42).fit(csr_matrix(X)) # ten dense dimensions
dense = svd.transform(csr_matrix(X))                # the dense vectors
explained = svd.explained_variance_ratio_           # share of variance per component
print("SVD explained variance (first five):", np.round(explained[:5], 3)) # report
```

实测：前五个成分的解释方差 0.212 / 0.206 / 0.180 / 0.125 / 0.045——说明低维就够用。完整程序 `l10_04_cooccurrence.py`。

GloVe：用比值编码语义 Ratios of Co-occurrence

关键观察

不是概率本身，而是两个概率的比值在编码语义：若" 冰" 与" 水" 在" 固态" 上比值很大、在" 气态" 上比值很小。

目标

让词向量满足

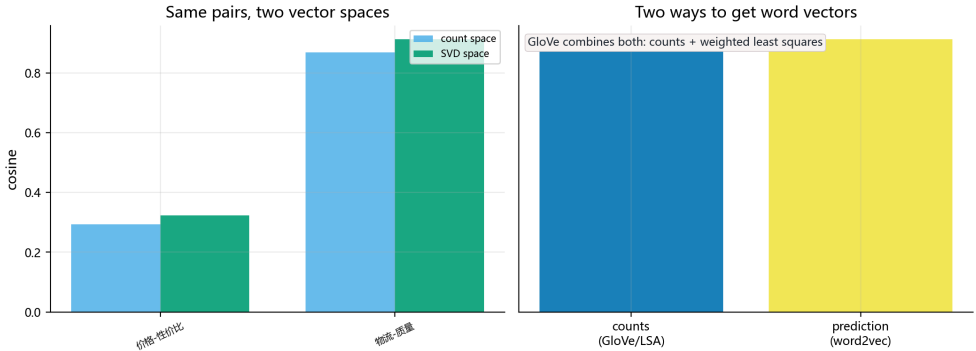
$$w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij},$$

用加权最小二乘拟合，权重是 $f(X_{ij})$。

GloVe = 计数的全局信息（像 LSA）+ 预测式的目标（像 word2vec）。本讲帧 66 把两派放在一张图上对比。

来源：比值观察与目标函数据 Pennington et al. (2014)。

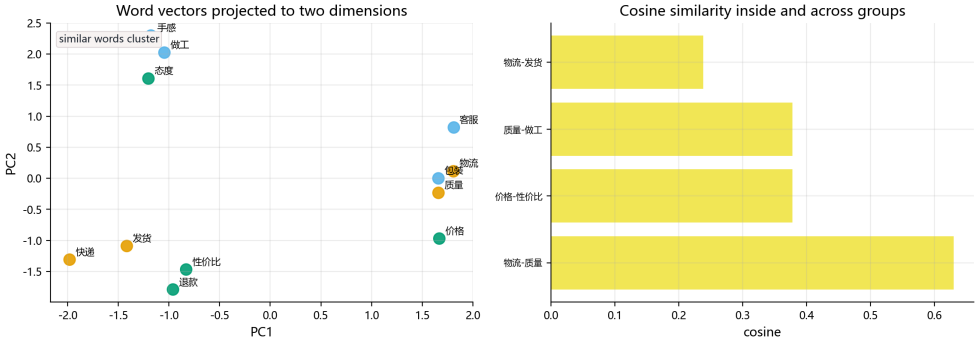
【样本 + 可视化】计数派与预测派



左：同一批词对在计数空间与 SVD 空间里的余弦相似度（排序一致，数值不同）；右：两大家族的示意——GloVe 把两者合并：用全局共现统计，拟合预测式的目标。

程序 `110_04_cooccurrence.py` ·对照表 `110_cooccurrence_similarity.csv`

【样本 + 可视化】词向量空间里，相似词挨在一起



左：16 个词投影到二维——物流/发货/快递/配送聚成一团，质量/做工/手感/包装一团，客服/售后/态度/退款一团，价格/性价比/便宜/贵一团；右：组内相似度明显高于组间（物流-质量 最低）。

程序 `110_03_word2vec.py` ·相似度表见控制台输出

内在评估：只评表示 Intrinsic Evaluation

两种任务

1. 相似度：给词对打人工分，看与余弦的相关 (Spearman)；
2. 类比：$a - b + c \approx d$，看检索对不对。

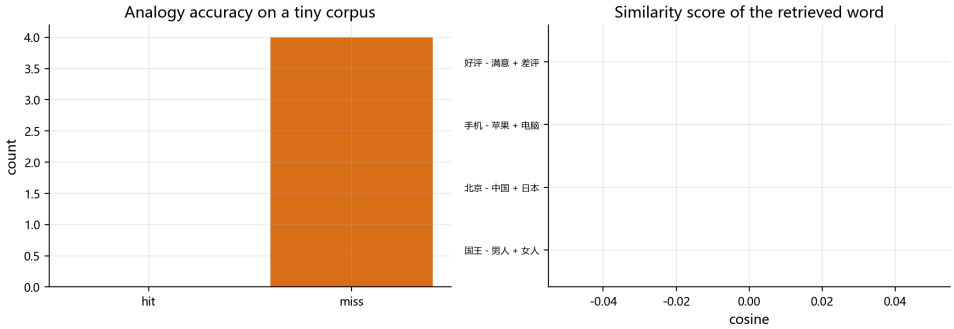
优点与陷阱

快、便宜、可比较；但可能过拟合到评估集——所以还要做外在评估（帧 70）。

课程讲义的重点之一：不同超参数下类比准确率差别很大（维度、窗口、语料规模）。本讲帧 69 给出一个小规模版本。

来源：评估方法据课程讲义第 112–122 页与 Mikolov et al. (2013b)。

【样本 + 可视化】类比评估：小语料上的成败



左：四组类比的命中情况；右：检索到的词的相似度分数。小语料上类比多半失败——这是正常的：类比需要大规模语料 + 足够维度。它的价值不在" 这个模型行不行"，而在提供一个可比较的标尺。

程序 `l10_03_word2vec.py` ·结果 `l10_analogies.csv`

外在评估：放进任务里评 Extrinsic Evaluation

做法

把词向量当作分类器的输入特征，比任务指标（准确率、F1）而不是相似度。

本讲对照

帧 75：TF-IDF + LR 与词向量平均 + MLP ——在这份语料上两者打平（0.967），说明表示不是瓶颈。

外在评估是最终标准：一个在相似度上更好的词向量，不一定让下游任务更好。报告里两个都要给。

来源：内在/外在评估的分工据课程讲义第 122 页；本讲对照来自 l10_05。

一词多义与上下文嵌入 Polysemy

问题

一个词一个向量，但" 苹果" 既是水果又是公司；" 便宜" 在" 价格便宜" 与" 便宜没好货" 里语气不同。

两条解法

1. 多原型：给一个词学多个向量（聚类上下文）；
2. 上下文嵌入：让向量依赖整句（ELMo/BERT，衔接 L9）。

今天的默认答案是第二条：同一个词在不同句子里得到不同向量。词向量没有消失，而是变成了上下文嵌入的初始化。

来源：多原型据 Huang et al. (2012) 与课程讲义第 123–126 页；上下文嵌入衔接 L9。

小结：词向量是”压缩的语义”

方法	怎么得到向量	代价
共现 + SVD	计数矩阵降维	内存大、需全局统计
word2vec	上下文预测 + 负采样	快、增量友好
GloVe	加权最小二乘拟合共现	需全局统计、稳定
上下文嵌入（L9）	整句过 Transformer	训练贵、推理贵

一句总结：词向量把”语义”变成了”几何”，于是相似度、类比、聚类都可以用向量运算来做。

来源：小结为本讲整理。

文本任务地图 The Task Map

任务	输入 → 输出	典型指标	本讲帧
分类 / 情感	文档 → 标签	准确率、F1	74–76
序列标注 / NER	序列 → 标签序列	实体级 F1	77–79
主题	文档集合 → 主题	纯度、困惑度	80–82
摘要	长文 → 短文	ROUGE	83
翻译	源语言 → 目标语言	BLEU	83
问答	问题 + 语料 → 答案	EM、F1	84

六个任务的输入输出结构不同，所以表示与评价都不同。本讲挑前三类做代码，后三类讲清原理与指标。

来源：任务与指标据课程大纲；编排为本讲安排。

为什么 TF-IDF 基线难超越 Why the Baseline Is Strong

三个原因

1. 文本分类往往由少数关键词决定（" 退款""坏了"）；
2. 线性模型在稀疏高维空间里不容易过拟合（有正则）；
3. 训练与推理都极快，便于迭代。

什么时候它不够

需要词序/否定/长距离依赖时（" 不是很满意"），或类别由组合决定时。

工程建议：先把 TF-IDF 调到最好（帧 55），再考虑词向量与微调。很多论文的" 提升" 其实只是没把基线调好。

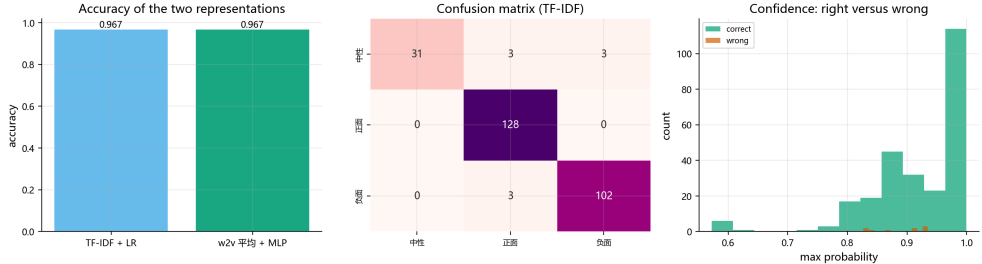
来源：三点原因据实践与文献；建议为本讲补充。

【代码】情感分类：两种表示对比

```python
from sklearn.feature_extraction.text import TfidfVectorizer # the sparse representation
from sklearn.neural_network import MLPClassifier      # the non-linear model
vec = TfidfVectorizer(analyzer=lambda toks: toks, max_features=2000) # tokens are already lists
Atr = vec.fit_transform(X_tr)                         # training matrix
Ate = vec.transform(X_te)                             # test matrix
lr = LogisticRegression(max_iter=2000).fit(Atr, y_tr) # the baseline
pred_lr = lr.predict(Ate)                             # test predictions
w2v = Word2Vec(sentences=list(X_tr), vector_size=40, window=4, min_count=1, sg=1, epochs=30, seed=20261111, workers=1) # train on the
      training split only
def average_vector(toks):                             # average the word vectors of a document
    vs = [w2v.wv[t] for t in toks if t in w2v.wv]      # the known vectors
    return np.mean(vs, axis=0) if vs else np.zeros(w2v.vector_size) # the average or zeros
Btr = np.vstack([average_vector(t) for t in X_tr])    # training matrix
mlp = MLPClassifier(hidden_layer_sizes=(32,), max_iter=800, random_state=42).fit(Btr, y_tr) # the MLP
pred_mlp = mlp.predict(Bte)                           # test predictions
```

实测：TF-IDF + LR 0.967（宏 F1 0.953）；词向量平均 + MLP 也是 0.967。两种表示打平——说明瓶颈在标注噪声（12%）而不是表示。完整程序 `l10_05_sentiment.py`。

【样本 + 可视化】情感分类：混淆矩阵与置信度



左：两种表示的准确率；中：混淆矩阵——主要错误在" 中性" 与" 正面/负面" 之间；右：模型置信度分布——判错的样本置信度偏低，这给了一个可用的" 人工复核" 阈值。

程序 `l10_05_sentiment.py` ·指标 `l10_sentiment_scores.csv` ·错误样例 `l10_sentiment_errors.csv`

序列标注与命名实体识别 Sequence Labelling and NER

BIO 标注

- B-ORG：实体的第一个 token；
- I-ORG：实体内部；
- O：实体外部。

两种实现

规则 + 词表（正则、后缀" 有限公司"）便宜可控；CRF/BiLSTM-CRF/预训练更鲁棒但要标注。

中文 NER 的一个实用技巧：用后缀词表（" 有限公司"" 股份"" 集团"）识别机构，再用金额与日期的正则补字段——本讲帧 78 就是这么做的。

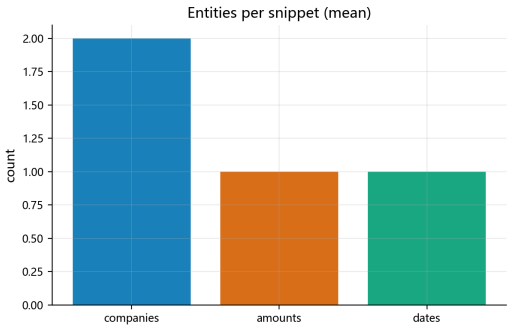
来源：BIO 与实现路线据 Jurafsky & Martin；中文技巧为本讲补充。

【代码】规则 + 词表的实体抽取

```python
COMPANY = re.compile(r"[\u4e00-\u9fa5]{2,12}(?:有限公司|股份有限公司|有限责任公司)") # company names
MONEY = re.compile(r"(?:人民币\s*)?[\d,\.]+\s*(?:万元|亿元|元)") # money expressions
DATE = re.compile(r"\d{4}\s*年\s*\d{1,2}\s*月\s*\d{1,2}\s*日|\d{4}-\d{2}-\d{2}|\d{4}\s*年\s*\d{1,2}\s*月") # dates
def extract(text):                                  # run all three extractors
    return {"company": COMPANY.findall(text), "money": MONEY.findall(text), "date": DATE.findall(text)} # the spans
for _, row in contracts.iterrows():                 # loop over the snippets
    found = extract(row["text"])                    # extract the entities
    rows.append({"doc_id": row["doc_id"], "text": row["text"], # the snippet
                 "companies": " | ".join(found["company"]), # extracted companies
                 "amounts": " | ".join(found["money"]), # extracted amounts
                 "dates": " | ".join(found["date"])})   # extracted dates
def bio_tag(text):                                  # build a BIO sequence
    spans = [(m.start(), m.end()) for m in COMPANY.finditer(text)] # company spans
    for start, end in spans:                        # mark the entity positions
        covered.update(range(start, end))           # add every character index
    tags.append((entity, "B-ORG" if start == min(s for s, _ in spans) else "I-ORG")) # B for the first
```

实测：60 条合同片段全部抽出 **公司、金额、日期**三类实体；BIO 序列逐字标注见 `l10_bio_example.csv`。完整程序 `l10_06_ner_rules.py`。

【样本 + 可视化】实体抽取的结果



One snippet, char-level BIO tags

BIO 标注示例：

本协议由华为技术有限公司与深圳市腾讯计算机系统有 → B-ORG
于2026 年 6 月签订，标的金额5,000 → O

左：每个片段平均抽出的实体数（公司近 2 个、金额与日期各 1 个）；右：一个片段的逐字 BIO 标注（红 =B，绿 =I，灰 =O）。规则法在模板化文本上几乎完美——真实合同里它会在嵌套与简称上出错。

程序 `l10_06_ner_rules.py` ·结果 `l10_ner_results.csv`

主题模型：从文档集合里提主题 Topic Models

LDA 的想法

每篇文档是若干主题的混合；每个主题是词上的分布。用生成过程拟合这两个分布。

两种做法

LDA（概率生成）；NMF（矩阵分解，L8 帧 93）。本讲两种都提，代码用 LDA。

评价：困惑度（perplexity）看拟合，纯度/人工可读性看质量。后者更重要——主题模型的成败常常在" 词读不读得懂"。

来源：LDA 据 Blei et al. (2003)；评价建议为本讲补充。

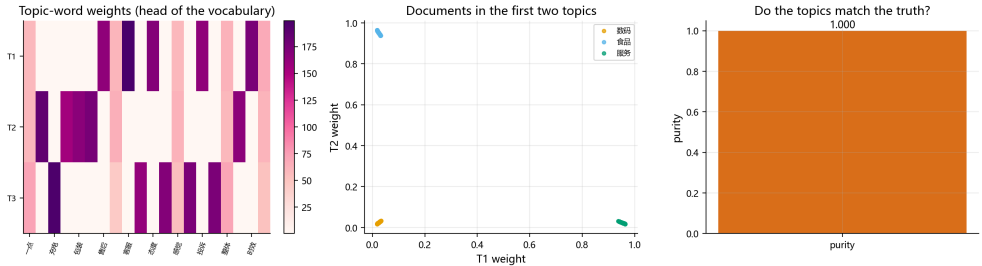
【代码】中文语料的 LDA

```python
from sklearn.decomposition import LatentDirichletAllocation # LDA
from sklearn.feature_extraction.text import CountVectorizer # the word counts
docs = [" ".join(cut(t)) for t in corpus["text"]]      # pre-tokenised documents
counts = CountVectorizer(max_features=200).fit_transform(docs) # the document-term matrix
lda = LatentDirichletAllocation(n_components=3, random_state=42, learning_method="batch").fit(counts) # three topics
for ti, comp in enumerate(lda.components_):            # loop over the topics
    order = np.argsort(comp)[::-1][:8]                  # the eight strongest words
    rows.append({"topic": "T%d" % (ti + 1), "top_words": " / ".join(vocab[i] for i in order)}) # record them
doc_topic = lda.transform(counts)                      # the weights per document
assigned = doc_topic.argmax(axis=1)                    # the dominant topic
purity = 0.0                                           # compute the cluster purity
for t in range(3):                                     # loop over the topics
    mask = assigned == t                               # documents assigned to this topic
    if mask.sum():                                     # avoid empty topics
        purity += float(corpus["true_topic"][mask].value_counts().iloc[0]) # the majority count
purity /= len(docs)                                    # normalise
```

实测：三主题在合成语料上的纯度 1.000；T1 = 客服/退款/时效/物流/态度/配送/投诉/售后 ——正好是" 服务" 主题。

完整程序 `l10_07_topics_zh.py`。

【样本 + 可视化】主题词与文档分布



左：主题-词权重矩阵（前 20 个词）；中：文档在前两个主题上的分布——三个簇清晰分开；右：纯度 1.000（与生成来源完全一致）。合成语料太干净，真实语料的纯度通常 0.6–0.8。

程序 l10_07_topics_zh.py ·主题词 l10_topics_zh.csv ·文档分配 l10_doc_topics_zh.csv

摘要与翻译：生成式任务的评价难题

摘要

抽取式（选句子）vs 生成式（重写）。抽取式可溯源，生成式更流畅。

翻译

从规则到统计到神经；今天主流是大模型直接翻。

指标：BLEU（翻译）与 ROUGE（摘要）都是n-gram 重合度，它们奖励"像参考译文/摘要"，不保证正确——所以生成式任务必须配人工评估（帧 86）。

来源：指标与局限据 Papineni et al. (2002)、Lin (2004) 与实践。

问答：检索 + 阅读 Question Answering

两条路线

1. 抽取式：从文档里找答案片段（SQuAD 风格）；
2. 生成式：让模型写答案（今天主流，配 RAG）。

关键问题

答案必须有出处。没有引用的答案在商业场景里不可用——这就是帧 92 的 RAG 与引用溯源。

评价：EM（完全匹配）与 F1（词级重叠）。它们对同义改写不友好，所以也要人工抽查。

来源：路线与指标据 Rajpurkar et al. (2016)；引用要求为本讲补充。

评价：指标、标注与人工评估

层面	工具	注意
分类	准确率、宏 F1、混淆矩阵	类别不平衡
序列标注	实体级 F1	边界算不算对
主题	纯度、困惑度、人工可读性	主题数怎么定
生成	BLEU / ROUGE + 人工	指标不等于正确
整体	Cohen's kappa	标注一致性（帧 90）

一个实用的报告模板：指标表 + 混淆矩阵 + 5 条错误样例 + 标注协议 + 一致性系数。这五样齐了，结论才可信。

来源：模板为本讲整理；与 L6–L9 的报告要求一致。

标注：一致性比数量更重要

为什么

如果两个人对同一批文本的判断都只有 70% 一致，那么模型 95% 的准确率无法解释——上限被标注定死了。

怎么做

1. 写标注指南（边界案例怎么处理）；
2. 双人标注一小批，算 kappa；
3. 低于 0.6 就改指南，而不是继续标。

这一节是" 结果与验证"评分项的核心。文本项目最容易犯的错，就是标了几千条却没检查过一致性。

来源：一致性门槛据 Landis & Koch (1977)；本节表述为本讲补充。

文本特有的泄漏 Text-Specific Leakage

三种泄漏

1. 同一文档的片段分到训练与测试；
2. 用测试集的词表建 TF-IDF；
3. 目标泄漏：文本里直接写了标签（" 这条是差评"）。

怎么防

按文档分组划分（GroupKFold）、只在训练集上 `fit` 向量器、检查是否有" 答案词"。

本讲的代码都遵循这条：向量器只在训练集上 fit（帧 55、75）。这一条比任何模型技巧都重要。

来源：泄漏类型据实践；" 按文档分组" 据 GroupKFold 文档。

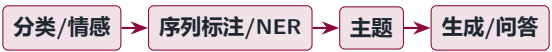
小结：任务决定表示与评价

三句话

分类先试 TF-IDF；抽取先试规则 + 词表；主题先看词读不读得懂。

共同要求

指标 + 错误样例 + 标注一致性 + 防泄漏。



下一部分：大模型改变了什么，以及 2026 年的实践做法。

来源：小结为本讲整理。

指令微调与少样本 Instruction Tuning and Few-Shot

三种使用方式

1. 零样本：只给指令；
2. 少样本：指令 + 几个示例；
3. 微调：用标注数据更新参数（L9）。

提示的结构

任务描述 + 输出格式 + 示例 + 输入。格式约束（" 只输出 JSON"）往往比措辞更重要。

少样本的代价：示例占上下文、结果对示例顺序敏感。所以要把提示当代码管理（帧 94）。

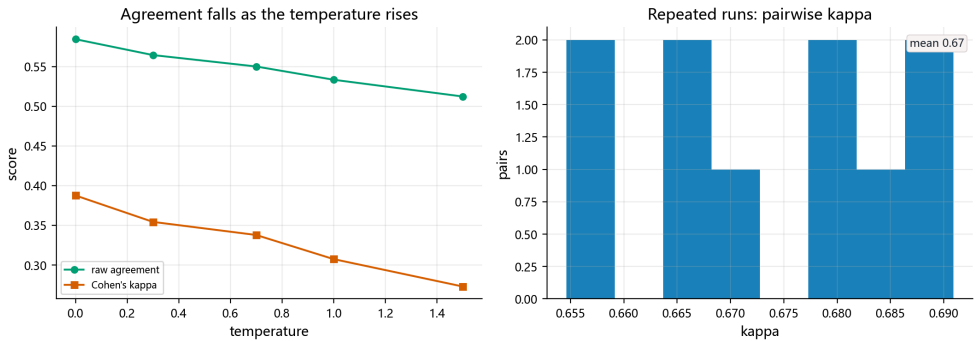
来源：少样本据 Brown et al. (2020)；指令微调据 Wei et al. (2022)。

【代码】用 kappa 核验 LLM 标注

```python
from sklearn.metrics import cohen_kappa_score         # agreement metric
human = reviews["label"].values                       # the human reference labels
def simulate_labels(texts, temperature, rng):         # a noisy rule-based "LLM"
    for text in texts:                                # loop over the documents
        pos_hit = sum(1 for w in positive if w in low)  # positive cue count
        neg_hit = sum(1 for w in negative if w in low)  # negative cue count
        if pos_hit > neg_hit:                         # the rule says positive
            base = "正面"                              # the base label
        if rng.random() < min(0.05 + temperature * 0.18, 0.55): # flip with probability rising in T
            base = str(rng.choice(["正面", "中性", "负面"])) # a random label
for temp in [0.0, 0.3, 0.7, 1.0, 1.5]:                # five temperature settings
    labels = simulate_labels(texts, temp, rng)        # generate the labels
    kappa = float(cohen_kappa_score(human, labels))   # agreement with the human labels
    rows.append({"temperature": temp, "agreement": round(agreement, 4), # the record
                 "cohen_kappa": round(kappa, 4)})     # the kappa
samples = [simulate_labels(texts, 0.7, rng) for _ in range(5)] # five repeated runs
```

实测：温度 0.0 → κ=0.387；温度 1.5 → κ=0.273；五次重复的平均两两 κ=0.674。完整程序 `l10_08_llm_agreement.py`。

【样本 + 可视化】LLM 标注的一致性曲线



左：温度越高，与人工的一致性越低（κ 从 0.39 降到 0.27）；右：五次重复运行的两两 κ 分布（均值 0.674）。重复采样的一致性衡量稳定性，与人工的一致性衡量正确性——两者都要报。

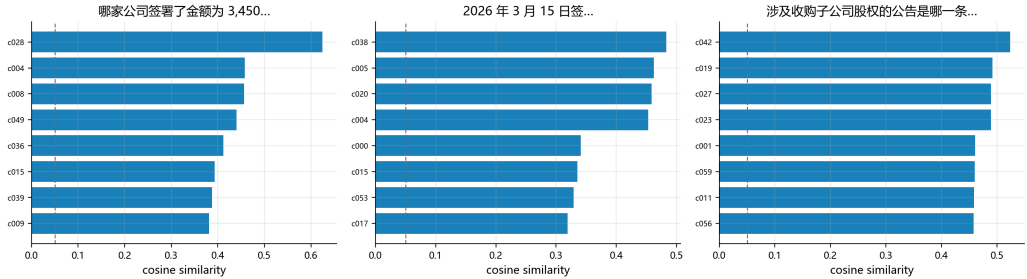
程序 `110_08_llm_agreement.py` ·表 `l10_llm_agreement.csv` / `l10_llm_pairwise_kappa.csv`

【代码】RAG：检索 + 生成 + 引用

```python
from sklearn.feature_extraction.text import TfidfVectorizer # the retrieval representation
from sklearn.metrics.pairwise import cosine_similarity # the similarity
docs = contracts["text"].tolist()                    # the passages
doc_ids = contracts["doc_id"].tolist()               # stable identifiers
corpus = [cut(d) for d in docs]                      # the tokenised corpus
vec = TfidfVectorizer().fit(corpus)                  # fit the retrieval model
matrix = vec.transform(corpus)                       # the document matrix
for qi, question in enumerate(questions):            # loop over the questions
    q = vec.transform([cut(question)])               # the query vector
    scores = cosine_similarity(q, matrix).ravel()    # similarity to every passage
    order = np.argsort(scores)[::-1][:3]             # the three best passages
    answer = " ".join("[%s] %s" % (doc_ids[i], docs[i]) for i in order if scores[i] > 0.05) # answer with citations
    rows.append({"question": question, "answer": answer, # the grounded answer
                 "top_id": doc_ids[order[0]], "top_score": round(float(scores[order[0]]), 4), # the best hit
                 "scores": [round(float(scores[i]), 3) for i in order]}) # the scores
```

实测：三条问题都检索到正确片段并附上[c0xx] 形式的引用；阈值 0.05 以下不引用——" 检索不到就说不确定"比编造安全。完整程序 l10_09_rag_citation.py。

【样本 + 可视化】检索分数与引用



每个面板是一条问题的检索分数（虚线是引用阈值）：排第一的片段被引用，其余低于阈值的被丢弃。RAG 的质量主要由检索决定——检索错了，生成再好也是错的。

程序 `l10_09_rag_citation.py` ·结果 `l10_rag_answers.csv` ·流程图 `fig_rag_pipeline.png`

提示词版本管理 Prompt Versioning

为什么要把提示当代码

改一个措辞，指标可能掉 10 个点；不记录版本，就无法复现也无法回滚。

最小做法

1. 提示存文件，命名带版本（v3）；
2. 固定评测集，每次改动跑回归；
3. 记录模型名、温度、日期。

三个记录项（模型 + 温度 + 提示版本）缺一，结果就不可复现。这与 L5 的模型卡是同一个纪律。

来源：版本管理做法据实践；与 L5 模型卡的关系为本讲补充。

【代码】领域词表如何改变分词

```python
terms = ["华为Mate60Pro", "比亚迪海豹", "vivoX100", "李宁矗科技", "ESG", "碳中和"] # the domain lexicon
def segment(text):                                  # the default segmentation
    return list(jieba.lcut(text))                   # cut with the current dictionary
before = [segment(s) for s in sentences]             # default cuts
for term in terms:                                  # add every domain term
    jieba.add_word(term)                            # to the dictionary
after = [segment(s) for s in sentences]             # cuts with the domain lexicon
for s, b, a in zip(sentences, before, after):       # loop over the sentences
    kept_before = [t for t in b if t in terms]      # terms surviving (default)
    kept_after = [t for t in a if t in terms]       # terms surviving (augmented)
    rows.append({"sentence": s, "default_cut": " / ".join(b), # the record
                 "augmented_cut": " / ".join(a),      # the augmented cut
                 "terms_before": len(kept_before), "terms_after": len(kept_after)}) # survival counts
test_terms = ["物流时效", "退换货", "性价比", "续航", "售后"] # five more business terms
```

实测：默认词典只保住 2 个领域术语，加入词典后保住 5 个。这 3 个术语的差别，会直接改变下游特征的维度。完整程序 `l10_10_domain_lexicon.py`。

隐私、成本与小结

三条纪律

1. 语料脱敏后再入模型（帧 16）；
2. 敏感数据用本地模型（衔接 L9 的量化端侧）；
3. 记录API 成本与调用量，写进报告。

2026 的现实

大模型让少样本成为默认起点；但评估、引用、合规这三件事，模型再强也不能替你完成。

本讲的落点：文本分析 = 表示 + 模型 + 评价 + 合规。前两个在变（词向量 → 大模型），后两个没变，且越来越重要。

来源：小结为本讲整理。

工具栈 Tool Stack

工具	擅长	本讲用在哪
jieba	中文分词、自定义词典	帧 47、95
scikit-learn	TF-IDF、LDA、指标、MLP	帧 55、75、81
Gensim	Word2Vec、相似度、类比	帧 62
spaCy	英文分词/词性/依存/NER	帧 77（原理）
Transformers	预训练模型与微调	衔接 L9 帧 87
LLM API	少样本、生成、RAG	帧 89–93

选型口诀：中文先上 jieba + 领域词典；基线用 sklearn；词向量用 Gensim；需要语义理解再上 Transformers / LLM。

来源：分工据各工具官方文档与本讲实践。

术语表 A–M Glossary A–M

中文	English
歧义	ambiguity
类比	analogy
标注	annotation
BIO 序列	BIO sequence
BIO 标注	BIO tagging
BLEU	BLEU
词袋	bag of words
Cohen's kappa	Cohen's kappa
引用溯源	citation tracing
共现矩阵	co-occurrence matrix
连接主义	connectionist approach
窗口	context window
上下文嵌入	contextual embedding
连续词袋	continuous bag of words (CBOW)
语料库	corpus
余弦相似度	cosine similarity
交叉验证	cross-validation
脱敏	de-identification
依存句法	dependency parsing
降维	dimensionality reduction
语篇	discourse

中文	English
分布假设	distributional hypothesis
文档-词矩阵	document-term matrix
领域词典	domain lexicon
嵌入矩阵	embedding matrix
外在评估	extrinsic evaluation
F1 分数	F1 score
少样本	few-shot
微调	fine-tuning
全半角	full-width and half-width
GloVe	global vectors (GloVe)
哈希技巧	hashing trick
信息抽取	information extraction
指令微调	instruction tuning
标注一致性	inter annotator agreement
内在评估	intrinsic evaluation
潜在狄利克雷分配	latent Dirichlet allocation (LDA)
泄漏	leakage
词形还原	lemmatisation
机器翻译	machine translation
形态学	morphology
多原型	multiple prototypes

本组 A–M 条，合表共 81 条；含符号、公式与一句话解释。见 17_glossary.csv 与站点词义页。

术语表 N–Z Glossary N–Z

中文	English
n 元语法	n-gram
命名实体识别	named entity recognition (NER)
负采样	negative sampling
归一化	normalisation
one-hot	one-hot encoding
未登录词	out-of-vocabulary (OOV)
词性标注	part-of-speech tagging
音系学	phonetics and phonology
一词多义	polysemy
预训练	pretraining
隐私	privacy
提示	prompt
提示版本管理	prompt versioning
问答	question answering
ROUGE	ROUGE
检索增强生成	retrieval-augmented generation (RAG)
中文分词歧义	segmentation ambiguity
语义学	semantics
情感分析	sentiment analysis
序列标注	sequence labelling

中文	English
奇异值分解	singular value decomposition (SVD)
跳字模型	skip-gram
稀疏	sparsity
统计方法	statistical approach
词干提取	stemming
停用词	stopword
摘要	summarisation
符号主义	symbolic approach
句法	syntax
TF-IDF	term frequency-inverse document frequency
文本分析	text analytics
文本作为数据	text as data
文本分类	text classification
主题模型	topic model
繁简转换	traditional-simplified conversion
变体	variation
分词	word segmentation
词向量	word vector / embedding
Word2vec	word2vec

本组 39 条，全表共 81 条，含符号、公式与一句话解释。见 17_glossary.csv 与站点讲义页。

结语：表示、评价、合规 Wrap-up

三句话

1. 文本分析的第一公里是分词与表示；
2. 词向量把语义变成几何，让相似变可算；
3. LLM 降低了门槛，但评价与合规仍是你的活。

自检

- 能说清" 研究生命起源" 的两种切法吗？
- 能解释 TF-IDF 为什么压高频词吗？
- 能算一个 kappa 并解释它吗？
- 能给你的答案加上引用吗？

谢谢！问题与项目讨论：Xieer.dai@szu.edu.cn；本讲代码、语料与术语表见 website/code/lecture10/。

来源：结语为本讲整理。